

포팅메뉴얼 - 외부서비스

1. GMS

1-1. GMS API 키 발급

- 환경변수에 등록할 GMS API 키 발급

1-2. 환경변수 등록

```
GMS_API_KEY=<GMS API 키>
GMS_BASE_URL=<GMS 기본 URL>
GMS_PATH_OPENAI=<GMS OPENAI PATH>
```

- 발급받은 GMS API KEY를 환경 변수로 등록한다.
- 사용 모델에 따라 GMS PATH를 환경 변수로 등록한다.

1-3. 서비스 연결

- `com.runningcity.global.client.gms`

```
package com.runningcity.global.client.gms;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;
import org.springframework.web.client.RestClientException;

import java.nio.charset.StandardCharsets;
import java.util.List;

@Slf4j
```

```

@Service
@RequiredArgsConstructor
public class OpenAiClient {
    @Value("${gms.api.key}")
    private String apiKey;

    @Value("${gms.api.base-url}")
    private String baseUrl;

    @Value("${gms.path.openai}")
    private String openaiPath;

    private final RestClient restClient = RestClient.create();

    private final ObjectMapper objectMapper;

    public String chat(String model, String system, String user) {
        ChatRequest body = new ChatRequest(
            model,
            List.of(
                new Message("system", system),
                new Message("user", user)
            ),
            0.7,
            4096
        );

        String jsonBody;
        try {
            jsonBody = objectMapper.writeValueAsString(body);
            log.info("GMS 요청 JSON: {}", jsonBody);
        } catch (JsonProcessingException e) {
            log.error("GMS 요청 JSON 직렬화 실패", e);
            throw new IllegalStateException("Failed to serialize OpenAI request body", e);
        }

        String url = baseUrl + "/" + openaiPath;
    }
}

```

```

ChatResponse response = restClient.post()
    .uri(url)
    .header("Content-Type", "application/json")
    .header("Authorization", "Bearer " + apiKey)
    .body(jsonBody)
    .retrieve()
    .onStatus(HttpStatusCode::isError, (req, res) → {
        String errBody;

        try {
            if (res.getBody() != null) {
                byte[] bytes = res.getBody().readAllBytes();
                errBody = bytes.length > 0
                    ? new String(bytes, StandardCharsets.UTF_8)
                    : "(empty body)";
            } else {
                errBody = "(null body)";
            }
        } catch (Exception e) {
            errBody = "(failed to read body: " + e.getMessage() +
")";
        }

        log.error("[OpenAiClient] HTTP {} from OpenAI/GMS. body
={}",
            res.getStatusCode(), errBody);

        throw new RestClientException("OpenAI/GMS returned erro
r: " +
            res.getStatusCode() + ", body=" + errBody);
    })
    .body(ChatResponse.class);

if (response == null
    || response.choices() == null
    || response.choices().isEmpty()
    || response.choices().get(0).message() == null) {

```

```

        log.error("Chat response is null");
        throw new IllegalStateException("Chat response is null");
    }

    return response.choices().get(0).message().content();
}

// DTO
public record Message(String role, String content) {}

public record ChatRequest(
    String model,
    List<Message> messages,
    Double temperature,
    Integer max_tokens
) {}

public record ChatResponse(List<Choice> choices) {
    public record Choice(Message message) {}
}
}

```

- 사용 모델 변경한 경우, Backend/src 의 `com.runningcity.global.client.gms` 와 `com.runningcity.report.service.AiReportService` 등을 요청/응답 가이드에 맞게 수정한다.

2. 카카오 지도 API

2-1. kakao developers 가입

- <https://developers.kakao.com> 접속 및 로그인
- 카카오 개발자 등록 절차 진행

2-2. 애플리케이션 생성

- 내 애플리케이션 → 애플리케이션 추가
- 앱 이름, 회사/개인명 추가

2-3. 앱 키 발급 및 서비스 등록

- 제공된 JavaScript 키 사용

1) API KEY 환경변수 등록

- `Frontend/.env`

```
VITE_KAKAO_MAP_APP_KEY=<API KEY>
```

2) 플랫폼 설정(Web 도메인 등록)

- 내 애플리케이션 → 플랫폼 → Web 플랫폼 추가
- 사이트 도메인, 개발 환경 도메인(localhost 등) 추가

2-4. SDK 로딩

- `Frontend/src/shared/lib/kakao/loadKakaoMapsSDK.ts`

```
import { ENV } from "@shared/config/env";
declare global { interface Window { kakao?: any } }

let kakaoLoadPromise: Promise<void> | null = null;

export function loadKakaoMapsSDK() {
  if (kakaoLoadPromise) return kakaoLoadPromise;

  kakaoLoadPromise = new Promise<void>((resolve, reject) => {
    if (window.kakao?.maps) { resolve(); return; }

    const existed = Array.from(document.scripts).find(s =>
      s.src.includes("dapi.kakao.com/v2/maps/sdk.js")
    );

    if (existed) {
      const wait = () => (window.kakao?.maps ? resolve() : setTimeout(wait, 50));
      wait();
      return;
    }
  });
}
```

```

    const s = document.createElement("script");
    s.async = true;
    s.src = `https://dapi.kakao.com/v2/maps/sdk.js?autoload=false&appkey=${ENV.KAKAO_MAP_APP_KEY}`;
    s.onload = () => window.kakao.maps.load(() => resolve());
    s.onerror = () => reject(new Error("Failed to load Kakao Maps SDK"));
    document.head.appendChild(s);
  });

  return kakaoLoadPromise;
}

```

- `Frontend/src/shared/lib/kakao/kakaoMapsProvider.tsx`

```

import { ReactNode, useEffect, useState } from "react";
import { loadKakaoMapsSDK } from "../loadKakaoMapsSDK";

export function KakaoMapsProvider({ children }: { children: ReactNode }) {
  const [ready, setReady] = useState(false);

  useEffect(() => {
    let active = true;
    // SSR 안전 가드
    if (typeof window === "undefined") return;

    loadKakaoMapsSDK()
      .then(() => active && setReady(true))
      .catch((e) => {
        console.error("[KakaoMaps] load failed", e);
        // 실패해도 앱 전체가 죽지 않도록 처리
      });

    return () => { active = false; };
  }, []);

```

```
return <>{children}</>;  
}
```

- `Frontend/src/app/providers/index.tsx`

```
import { KakaoMapsProvider } from '@shared/lib/kakao/kakaoMapsProvider';  
  
...  
  
export function Providers({ children }: ProvidersProps) {  
  return (  
    <QueryClientProvider client={qc}>  
      <KakaoMapsProvider>  
        {children}  
      </KakaoMapsProvider>  
    </QueryClientProvider>  
  );  
}
```

3. Crashlytics

3-1. Firebase 프로젝트 생성

- Firebase Console 페이지 접속
- 프로젝트 이름 입력, 기타 설정 확인 후 프로젝트 생성

3-2. 앱 생성

- 생성한 Firebase 프로젝트 페이지로 이동
- 앱 추가 → 플랫폼 선택 (Android)
- 안드로이드 패키지 이름 입력(ApplicationId)
- 제공된 `google-services.json` 다운로드 후 `app` 모듈 `src` 하위에 삽입
- Firebase SDK 추가
 - `libs.versions.toml`

```

[versions]
# crashlytics
firebaseBom = "34.5.0"
firebaseCrashlytics = "19.2.0"
firebaseCrashlyticsGradle = "3.0.6"
googleServices = "4.4.4"

[libraries]
# crashlytics
firebaseBom = { module = "com.google.firebase:firebase-bom", version.ref = "firebaseBom" }
firebaseCrashlytics = { module = "com.google.firebase:firebase-crashlytics-ktx", version.ref = "firebaseCrashlytics" }

[plugins]
googleServices = { id = "com.google.gms.google-services", version.ref = "googleServices" }
firebaseCrashlyticsPlugin = { id = "com.google.firebase.crashlytics", version.ref = "firebaseCrashlyticsGradle" }

```

- app 모듈의 `build.gradle.kts`

```

plugins {
    ...
    alias(libs.plugins.googleServices)
    alias(libs.plugins.firebaseCrashlyticsPlugin)
}

...

dependencies {
    ...
    // crashlytics
    implementation(platform(libs.firebaseBom))
    implementation(libs.firebaseCrashlytics)
}

```


- Sync

3-3. 앱 크래시 테스트

```
val crashButton = Button(this).apply {
    text = "Test Crash"
    setOnClickListener {
        throw RuntimeException("Test Crash") // Force a crash
    }
}

setContentView(
    LinearLayout(this).apply {
        orientation = LinearLayout.VERTICAL
        addView(
            crashButton,
            LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.WRAP_CONTENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            )
        )
    }
)
```

- 화면에 버튼 삽입 후 클릭 테스트
- Firebase Console에서 확인